

SERVICIO NACIONAL DE APRENDIZAJE

PROGRAMA
TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE

FICHA
3013188

NOMBRE DE APRENDICES
SEBASTIÁN SCARPETTA GONZÁLEZ
WILBER ALBERTO VARGAS GÓMEZ

INSTRUCTOR
CLAUDIA BELTRÁN RODRÍGUEZ

TALLER SOBRE COMPONENTES FRONT-END

NEIVA – HUILA
2025

INTRODUCCIÓN

En el desarrollo de aplicaciones modernas, los componentes front-end son la base para construir interfaces interactivas, dinámicas y eficientes. En este contexto, React se ha consolidado como una de las bibliotecas más populares para el desarrollo de interfaces de usuario debido a su enfoque declarativo, su flexibilidad y su facilidad para manejar el estado de los componentes. El presente taller tiene como propósito comprender los fundamentos de React y algunos de sus elementos más relevantes, como JSX, las clases y los eventos, además de representar gráficamente su estructura conceptual. A través de este análisis se busca fortalecer las competencias del aprendiz en el desarrollo de aplicaciones web modernas y escalables.

1. DIFERENCIAS ENTRE REACT Y JSX

Aunque React y JSX suelen mencionarse juntos, representan conceptos distintos dentro del desarrollo con esta biblioteca. React es una biblioteca de JavaScript creada por Facebook para construir interfaces de usuario basadas en componentes. Su función principal es manejar la vista dentro del modelo MVC (Modelo-Vista-Controlador), permitiendo que cada parte de la interfaz sea modular, reutilizable y fácil de mantener. React utiliza un DOM virtual (Virtual DOM) que mejora el rendimiento al actualizar solo los elementos que cambian en la interfaz.

Por otro lado, JSX (JavaScript XML) es una extensión de la sintaxis de JavaScript que

permite escribir código similar a HTML dentro de los archivos de React. Su objetivo es facilitar la lectura y escritura de componentes, haciendo que el código sea más comprensible. JSX no es entendido directamente por el navegador, por lo que debe ser transformado en código JavaScript puro mediante herramientas como Babel.

En resumen, React es la biblioteca que permite crear la interfaz, mientras que JSX es la sintaxis que facilita la construcción de esos componentes. React puede funcionar sin JSX, pero JSX hace el proceso más intuitivo y legible.

2. ¿QUÉ SON LAS CLASES EN REACT?

En las versiones iniciales de React, las clases eran el principal método para definir componentes. Una clase en React es una estructura basada en la programación orientada a objetos que hereda de `React.Component` y que contiene un método obligatorio llamado `render()`, el cual devuelve los elementos que se mostrarán en la interfaz.

Las clases permiten manejar el estado interno (state) y los ciclos de vida del componente, como `componentDidMount()` o `componentDidUpdate()`, funciones que controlan el comportamiento del componente en diferentes etapas. Sin embargo, desde la introducción de los hooks en React 16.8, muchos desarrolladores han migrado al uso de componentes funcionales, que logran los mismos resultados con menos código y mayor simplicidad.

3. PRINCIPALES EVENTOS DE REACT

Los eventos en React son acciones o sucesos que ocurren como resultado de la interacción del usuario con la interfaz, tales como clics, movimientos del ratón, escritura en un campo de texto o envío de un formulario. React maneja los eventos de forma muy similar a JavaScript, pero con algunas diferencias importantes.

- `onClick`: se activa al hacer clic sobre un elemento.
- `onChange`: se usa para detectar cambios en un campo de entrada.
- `onSubmit`: se ejecuta cuando un formulario es enviado.
- `onMouseOver` / `onMouseOut`: se activan al pasar o quitar el puntero del ratón sobre un elemento.

React implementa un sistema de eventos sintéticos llamado `SyntheticEvent`, que unifica los diferentes comportamientos de los navegadores, asegurando compatibilidad y eficiencia.

4. MAPA CONCEPTUAL DE REACT

Mapa conceptual de React

Virtual DOM, reconciliación y rendimiento

Propósito y función

- Virtual DOM es una representación ligera de la UI que React usa para calcular cambios mínimos al DOM real.
 - Reconciliación compara versiones previas y actuales del Virtual DOM para aplicar actualizaciones eficientes.
 - Reduce operaciones costosas en el DOM real, mejorando rendimiento en aplicaciones interactivas.
 - Ejemplo: actualización de un solo elemento en la lista sin re-renderizar todo el árbol.
- Keys en listas para ayudar al algoritmo de reconciliación a identificar elementos estables.
- Evitar mutaciones directas del estado y usar actualizaciones inmutables para permitir detección de cambios.
 - Memoización con `React.memo` y `useMemo` para evitar renders innecesarios en componentes pesados.
 - División de código (code-splitting) y lazy loading para reducir tiempo de carga inicial.

Implementación y buenas prácticas de rendimiento

Eventos y manejo de entradas del usuario

Propósito y función

- Eventos permiten responder a acciones del usuario como clics, cambios en formularios y envíos.
 - Normalizan nomenclatura en camelCase (`onClick`, `onChange`) y pasan funciones manejadoras como props.
 - Facilitan validación, prevención de comportamiento por defecto y actualización de estado en tiempo real.
 - Ejemplo: `onSubmit` en formulario para prevenir reload y procesar datos con JavaScript.
- `onClick`: enlaza funciones handler que actualizan estado o navegan entre vistas.
- `onChange`: captura entradas de formularios, actualizando estado controlado para inputs y selects.
 - `onSubmit`: manejar envío de formularios con `event.preventDefault()` y validación asíncrona.
 - Uso de handlers definidos con `useCallback` para optimizar re-renderizados y performance.

Implementación y patrones comunes

Estado (State) y manejo de efectos

Propósito y función

- Estado guarda datos locales que determinan la representación y pueden cambiar con interacciones del usuario.
 - Permite que el componente se re-renderice automáticamente cuando el estado cambia.
 - Efectos (side effects) gestionan suscripciones, fetch de datos y sincronización con APIs externas.
 - Ejemplo: `useState` para contador; `useEffect` para cargar datos al montar.
- `useState`: declara variables de estado y función actualizadora dentro de componentes funcionales.
- `useEffect`: ejecuta lógica tras renderizados, reemplazando métodos de ciclo de vida como `componentDidMount`.
 - Dependencias en `useEffect` para controlar cuándo se ejecuta y evitar bucles infinitos.
 - Uso de `cleanup` en `useEffect` para cancelar timers o suscripciones al desmontar.

Implementación con Hooks y ciclo de vida

Sintaxis JSX y su propósito en componentes

Propósito y función

- Sintaxis extendida que permite escribir estructura similar a HTML dentro de JavaScript para definir la UI de manera declarativa y legible.
- Facilita la composición de elementos y componentes mediante una sintaxis familiar para desarrolladores web.
- Soporta expresiones JavaScript dentro de llaves, permitiendo lógica y renderizado condicional directamente en la plantilla.
- Ejemplo: `return <div>{title}</div>` para renderizar variables dentro de la estructura JSX.

Implementación y características clave

- Transpilación necesaria mediante Babel o herramientas modernas para convertir JSX a llamadas `React.createElement`.
- Permite atributos y props, que se pasan a componentes como propiedades configurables.
- Fragmentos (`<< />`) para agrupar elementos sin añadir nodos DOM adicionales.
- Uso de keys en listas para mejorar el rendimiento y evitar advertencias durante re-renderizados.

Componentes: tipos y composición

Propósito y función

- Componentes encapsulan UI y lógica reutilizable, promoviendo separación de responsabilidades y composición.
- Permiten construir interfaces complejas combinando componentes más pequeños (composición).
- Soportan props para parametrizar comportamiento y presentación desde el componente padre.
- Ejemplo: `<Button onClick={handle}>Enviar</Button>` muestra comunicación padre-hijo mediante props.

Implementación: funcionales y basados en clases

- Componentes funcionales: funciones puras que retornan JSX, ahora con Hooks para estado y efectos.
- Componentes de clase: extendían `React.Component` y manejaban estado y ciclo de vida mediante métodos.
- Migración común: convertir clases a funciones con Hooks para simplificar código y mejorar reusabilidad.
- Ejemplo: `function MyComp() { const [x, setX] = useState(0); return <div>{x}</div>; }`

CONCLUSIÓN

React se ha consolidado como una herramienta esencial para el desarrollo moderno de aplicaciones web, al ofrecer una estructura clara basada en componentes reutilizables. Comprender la diferencia entre React y JSX permite escribir código más legible y eficiente, mientras que conocer las clases y los eventos amplía la capacidad del desarrollador para crear interfaces interactivas y funcionales.

BIBLIOGRAFÍA

<https://zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/Tecnologia/228118/Contenido/OVA/CF31/index.html#/>

LenguajeJS. (s. f.). JSX: El HTML de React. Recuperado de <https://lenguajejs.com/react/componentes/jsx/>

ESIC Business & Marketing School. (2025, marzo). React: Definición, características y ejemplos de uso. Recuperado de <https://www.esic.edu/rethink/tecnologia/react-que-es-c>

freeCodeCamp Español. (2024). Aprende React desde cero – Curso de React con proyectos. Recuperado de <https://www.freecodecamp.org/espanol/news/aprende-react-desde-cero-curso-de-react-con-proyectos/>